

Procvičovací zadání: Rezervační systém kurzů

Středně pokročilé zadání k procvičení OOP v Javě - o trochu těžší než základní polymorfismus, ale stále bez návrhových vzorů a složitých algoritmů.

CÍL ÚKOLU

Vytvořte konzolovou aplikaci, která simuluje jednoduchý rezervační systém volnočasových kurzů. Systém bude umět evidovat různé typy kurzů, přidávat účastníky, počítat cenu kurzu a vypsát přehled rezervací.

Zadání je navrženo tak, aby procvičilo hlavně dědičnost, rozhraní, polymorfismus, kolekce a jednoduchou implementaci metod equals() a hashCode().

TÉMA

Městské centrum pořádá různé kurzy. Některé jsou sportovní, jiné vzdělávací. Každý kurz má název, lektora, kapacitu a seznam přihlášených účastníků. U některých kurzů se cena počítá jinak podle typu kurzu.

POŽADOVANÉ TŘÍDY A ROZHRANÍ

1. Třída Participant

- Atributy: id (String), name (String), email (String).
- Vytvořte konstruktory a gettery.
- Přepište metodu toString(), aby vypisovala např. P001 - Jana Nováková (jana@email.cz).
- Přepište metody equals() a hashCode() tak, aby se účastníci porovnávali pouze podle id.

2. Rozhraní Discountable

Vytvořte rozhraní Discountable, které bude obsahovat metodu:

```
double getDiscount();
```

Metoda vrátí slevu v procentech. Například hodnota 0.10 znamená slevu 10 %.

3. Abstraktní třída Course

Vytvořte abstraktní třídu Course s atributy:

- code - kód kurzu, např. JAVA01
- title - název kurzu
- lecturer - jméno lektora
- capacity - maximální počet účastníků
- basePrice - základní cena kurzu
- participants - kolekce účastníků; použijte HashSet<Participant>, aby nešlo přidat stejného účastníka dvakrát.

Třída bude obsahovat tyto metody:

- boolean addParticipant(Participant p) - přidá účastníka pouze tehdy, pokud kurz není plný a účastník ještě není přihlášen.

- boolean removeParticipant(String participantId) - odebere účastníka podle id.
- boolean isFull() - vrátí true, pokud je kurz plný.
- int getFreePlaces() - vrátí počet volných míst.
- abstract double calculatePrice() - každá podtřída určí cenu jinak.
- String getInfo() - vrátí stručný popis kurzu včetně typu kurzu, ceny a obsazenosti.

PODTŘÍDY KURZU

4. Třída SportCourse extends Course

- Přidejte atribut equipmentIncluded (boolean), který říká, zda je v ceně zapůjčení vybavení.
- Pokud je vybavení v ceně, metoda calculatePrice() vrátí basePrice + 300.
- Jinak vrátí pouze basePrice.
- Metoda getInfo() bude obsahovat text Sportovní kurz a informaci, zda je vybavení v ceně.

5. Třída LanguageCourse extends Course implements Discountable

- Přidejte atribut language (String), např. Angličtina, Němčina.
- Metoda getDiscount() vrátí 0.10, tedy 10% slevu.
- Metoda calculatePrice() vrátí cenu po slevě: $\text{basePrice} * (1 - \text{getDiscount}())$.
- Metoda getInfo() bude obsahovat text Jazykový kurz, jazyk a informaci o slevě.

6. Třída ArtCourse extends Course

- Přidejte atribut materialCost (double), který představuje cenu materiálu.
- Metoda calculatePrice() vrátí $\text{basePrice} + \text{materialCost}$.
- Metoda getInfo() bude obsahovat text Umělecký kurz a cenu materiálu.

SPRÁVCE KURZŮ

7. Třída CourseManager

Vytvořte třídu CourseManager, která bude spravovat seznam všech kurzů. Použijte ArrayList<Course>.

Třída bude obsahovat metody:

- void addCourse(Course course) - přidá kurz do seznamu.
- Course findCourseByCode(String code) - najde kurz podle kódu; pokud neexistuje, vrátí null.
- boolean enroll(String courseCode, Participant participant) - najde kurz a pokusí se přihlásit účastníka.
- void printAllCourses() - vypíše informace o všech kurzech.
- void printOnlyAvailableCourses() - vypíše pouze kurzy, které ještě nejsou plné.

POŽADAVKY NA MAIN

- V metodě main vytvořte alespoň 3 různé kurzy: jeden sportovní, jeden jazykový a jeden umělecký.
- Vytvořte alespoň 5 účastníků.

- Přihlaste účastníky do kurzů tak, aby se otestovalo běžné přihlášení, zaplnění kurzu i pokus o duplicitní přihlášení stejného účastníka.
- Vypište všechny kurzy pomocí jedné kolekce typu `ArrayList<Course>`, aby bylo vidět polymorfní volání metody `getInfo()`.
- Otestujte odebrání účastníka z kurzu podle id.
- Otestujte výpis pouze dostupných kurzů.

DOPORUČENÝ POSTUP

- Nejprve vytvořte třídu `Participant` a správně vyřešte `equals()` a `hashCode()`.
- Poté vytvořte abstraktní třídu `Course` a otestujte přidávání účastníků do `HashSetu`.
- Až potom doplňte jednotlivé podtřídy kurzů.
- Nakonec vytvořte `CourseManager` a metodu `main`.

PŘÍKLAD MOŽNÉHO VÝSTUPU

```

=== REZERVAČNÍ SYSTÉM KURZŮ ===
Přidávám účastníky...
Jana byla přihlášena do kurzu JAVA01: true
Jana byla znovu přihlášena do kurzu JAVA01: false

=== VŠECHNY KURZY ===
JAVA01 - Jazykový kurz: Angličtina, lektor: Novotná, cena: 2700.0 Kč, obsazeno 2/3, sleva 10 %
FIT01 - Sportovní kurz: Jóga pro začátečníky, lektor: Svoboda, cena: 1800.0 Kč, obsazeno 4/4, vybavení v ceně: ne
ART01 - Umělecký kurz: Keramika, lektor: Malá, cena: 2400.0 Kč, obsazeno 1/5, materiál: 400.0 Kč

=== DOSTUPNÉ KURZY ===
JAVA01 - volná místa: 1
ART01 - volná místa: 4

Odebrání účastníka P003 z kurzu FIT01: true

```

CO SE TÍMTO ZADÁNÍM PROCVIČUJE

Oblast	Kde se objeví v zadání
Dědičnost	<code>SportCourse</code> , <code>LanguageCourse</code> a <code>ArtCourse</code> dědí z <code>Course</code> .
Abstraktní třída	<code>Course</code> obsahuje společné atributy a abstraktní metodu <code>calculatePrice()</code> .
Rozhraní	<code>LanguageCourse</code> implementuje <code>Discountable</code> .
Polymorfismus	Kurzy jsou uloženy v <code>ArrayList<Course></code> a volá se <code>getInfo()</code> .
Kolekce	<code>ArrayList</code> pro kurzy, <code>HashSet</code> pro účastníky.
<code>equals/hashCode</code>	<code>Participant</code> se porovnává podle id, aby <code>HashSet</code> poznal duplicitu.

BONUS - pouze pokud máte hotovo

- Přidejte metodu `printParticipants(String courseCode)`, která vypíše všechny účastníky konkrétního kurzu.
- Umožněte v `CourseManager` spočítat celkový počet přihlášených účastníků ve všech kurzech.
- U jazykového kurzu přidejte úroveň kurzu, např. A1, B1, C1.